

Product Service

Code-wise request flow through JWT authentication, gateway authorization, routing, logging, rate limiting, caching, and ProductController.

Request followed in this guide

GET `http://localhost:8080/api/products`
Authorization: Bearer <JWT>

SecurityConfig: validate and authorize

|
v

application.yml: choose product-service route

|
v

Gateway filters: log, limit, cache, rewrite

|
v

ProductController: create and return product data

Most important distinction

jwtDecoder() is a bean factory called during startup. **decoder.decode(token)** is called by Spring Security when an actual request arrives.

Prepared from the current source under D:\OBPwork. File names and line numbers in this document refer to that source snapshot.

1. How to read the call flow

The request crosses two kinds of code. Your project configures the behavior; Spring framework classes execute much of the runtime pipeline.

Label	Meaning	Examples
YOUR CODE	A method, annotation, or YAML line in this project.	SecurityConfig, RequestLoggingFilter, ProductController
SPRING	Dependency code automatically invoked by WebFlux, Spring Security, or Gateway.	AuthenticationWebFilter, JwtReactiveAuthenticationManager

Two phases

Phase	What happens	Does a request token exist?
Startup	Spring creates and connects decoder, converter, filters, and route definitions.	No
Request time	The stored objects extract, validate, authorize, route, and return the request.	Yes

Source map

File	Relevant lines	Responsibility
ApiGatewayApplication.java	12-13	Starts API Gateway
SecurityConfig.java	33-51	Builds security filter chain
SecurityConfig.java	55-66	Creates HS256 secret key
SecurityConfig.java	77-82	Creates JWT decoder bean
SecurityConfig.java	86-94	Creates JWT roles converter
application.yml	28-40	Product route and route filters
RequestLoggingFilter.java	26-39	Correlation ID and request logging
RateLimitConfig.java	15-23	Username or IP rate-limit key
ProductController.java	16, 24-41	Product endpoint and response

Rule of thumb

SecurityConfig decides whether the request may continue. application.yml decides where it goes. ProductController decides what data is returned.

2. Startup wiring: how .jwt() reaches the decoder

1

Gateway starts

ApiGatewayApplication.java:12-13

SpringApplication.run(...) starts the context and scans configuration classes.

Next: SecurityConfig beans are created.

2

SecretKey bean is created

SecurityConfig.java:55-66

Line 58 decodes the Base64 secret. Line 66 returns a HmacSHA256 SecretKey.

Next: The key is injected into jwtDecoder(...).

3

ReactiveJwtDecoder bean is created

SecurityConfig.java:77-82

Lines 78-80 build NimbusReactiveJwtDecoder for HS256. Line 81 attaches issuer and standard time validators. Line 82 returns the decoder object.

Next: Spring stores the returned decoder object.

4

JWT converter bean is created

SecurityConfig.java:86-94

Lines 88-89 configure claim roles and prefix ROLE_. Line 94 returns the converter object.

Next: The converter is injected into securityWebFilterChain(...).

5

Resource-server JWT support is registered

SecurityConfig.java:49-50

oauth2ResourceServer(...).jwt(...) tells Spring to process Bearer JWTs and use the supplied converter after validation.

Next: build() assembles the runtime filters.

6

SecurityWebFilterChain is built

SecurityConfig.java:51

Spring finds the ReactiveJwtDecoder bean, creates JwtReactiveAuthenticationManager, and places AuthenticationWebFilter in the chain.

Next: The chain waits for requests.**STARTUP ONLY**

```
.jwt(configuration)
  -> saves JWT settings
.build()
  -> connects AuthenticationWebFilter
  -> JwtReactiveAuthenticationManager
  -> stored ReactiveJwtDecoder
  -> stored JWT converter
```

Why no decode() appears after .jwt()

There is no token during startup. .jwt() registers instructions; build() creates the machinery. The machinery calls decode(token) later, when a request supplies a token.

3. Request time: token extraction and validation

Incoming request

GET /api/products

Authorization: Bearer eyJ...

1

Request enters the built security chain

SecurityConfig.java:51 -> SPRING

WebFilterChainProxy selects the SecurityWebFilterChain created during startup.

Next: AuthenticationWebFilter.filter(exchange).

2

Bearer token is extracted

SPRING: ServerBearerTokenAuthenticationConverter.convert(exchange)

The Authorization header is read and the value after Bearer is placed in an unauthenticated BearerTokenAuthenticationToken.

Next: JwtReactiveAuthenticationManager.authenticate(...).

3

Authentication manager receives the token

SPRING: JwtReactiveAuthenticationManager.authenticate(authentication)

It reads the token string from authentication credentials.

Next: Calls decode(token) on the stored decoder bean.

4

Decoder validates the JWT

Object returned by SecurityConfig.java:82

NimbusReactiveJwtDecoder checks JWT structure, HS256 signature, expiration, not-before time, and issuer.

Next: Valid: return Mono<Jwt>. Invalid: authentication error and 401.

REQUEST-TIME SPRING CALLS

```
AuthenticationWebFilter.filter(exchange)
  -> ServerBearerTokenAuthenticationConverter.convert(exchange)
  -> JwtReactiveAuthenticationManager.authenticate(authentication)
  -> storedDecoder.decode(token)
  -> storedConverter.convert(validatedJwt)
```

Exact connection to your code

SecurityConfig.java:77 is the factory method. SecurityConfig.java:82 returns the decoder object. At request time, Spring calls decode(token) on that returned object; it does not call the factory method again.

4. Valid JWT -> authenticated user -> authorization

A. Validated JWT goes to the stored converter

```
{
  "sub": "user",
  "roles": ["USER"],
  "iss": "demo-api-gateway"
}
```

After `decode(token)` succeeds, `JwtReactiveAuthenticationManager` calls the converter returned from `SecurityConfig.java:94`.

```

                JWT claim roles = [USER]
                    |
                    v
        SecurityConfig.java:88 reads claim 'roles'
                    |
                    v
        SecurityConfig.java:89 adds prefix 'ROLE_'
                    |
                    v
        JwtAuthenticationToken: user, authenticated=true, [ROLE_USER]
```

B. Spring stores Authentication for this request

The authenticated object is placed in the reactive security context. From this point, `exchange.getPrincipal()` can produce the username user.

C. Authorization rules are evaluated in order

SecurityConfig line	Rule	Does GET /api/products match?
42-43	Public auth and actuator URLs	No
44	POST /api/orders/** requires ADMIN	No
45	/api/products/** and /api/orders/** require authentication	Yes
46-47	Admin actuator, then deny everything else	Not reached

Result at SecurityConfig.java:45

`authenticated()` checks the authenticated result created above. It does not itself decode the token. Valid user or admin JWT: continue. Missing or invalid JWT: 401 and stop.

Token case	Result	Does routing run?
Valid	Authentication created; line 45 permits request	Yes
Missing	Protected request remains anonymous; 401	No
Invalid signature	Decoder produces authentication error; 401	No
Expired or wrong issuer	Validator produces authentication error; 401	No

5. Authorization success -> product route -> logging

1

Product route predicate matches

application.yml:28-31

The path `/api/products` matches `Path=/api/products/**`. Route ID becomes `product-service`.

Next: Read the destination URI.

2

Destination is selected

application.yml:29

Local default is `http://localhost:8081`. In Docker, `PRODUCT_SERVICE_URL` normally resolves to `http://product-service:8081`.

Next: Gateway filters execute around the route.

3

Logging filter starts

RequestLoggingFilter.java:26-32

Line 27 records start time. Line 28 calls `correlationId(exchange)`. Lines 29-32 add the correlation header and create an updated exchange.

Next: Read the authenticated Principal.

4

Username is read

RequestLoggingFilter.java:34-36

`getPrincipal()` supplies the authenticated Principal. `map(Principal::getName)` produces the username; `defaultIfEmpty` uses anonymous when absent.

Next: Continue to the next gateway filter.

5

Filter chain continues

RequestLoggingFilter.java:37-39

`chain.filter(updatedExchange)` invokes the remaining route filters. `doFinally` schedules logging after completion.

Next: Strip prefix, rate limit, cache, and route.

SECURITY MATCHER VS ROUTE PREDICATE

SecurityConfig.java:45

`/api/products/** -> Is the user allowed?`

application.yml:31

`/api/products/** -> Which service receives it?`**Do not combine these responsibilities**

The identical-looking path patterns belong to different systems. Security grants access first; Gateway routing then selects `product-service`.

6. Meaning of map(Principal::getName)

```
return exchange -> exchange.getPrincipal()
    .map(Principal::getName)
    .switchIfEmpty(Mono.fromSupplier(() -> {
        // return client IP
    }));
```

Line-by-line meaning: RateLimitConfig.java:16-23

Expression	Type / result	Meaning
exchange.getPrincipal()	Mono<Principal>	May asynchronously emit the authenticated Principal, or be empty.
.map(...)	Transforms an emitted value	Runs only if the Mono contains a Principal.
Principal::getName	Principal -> String	Calls getName() on the Principal.
.switchIfEmpty(...)	Fallback Mono<String>	Uses the client IP when no Principal was emitted.

Method reference expanded

```
.map(Principal::getName)

// Exactly equivalent to:
.map(principal -> principal.getName())
```

Mono<Principal> emits Principal(name='user')

|
v

map calls principal.getName()

|
v

Mono<String> emits 'user'

|
v

RequestRateLimiter uses 'user' as the Redis key

What map does not do

It does not validate the JWT, loop over users, query a database, or change the Principal. JWT validation already created the Principal; map only transforms Principal into its name.

Example: authenticated JWT subject user -> Principal.getName() returns user -> Redis rate-limit bucket is associated with user. If no Principal exists, switchIfEmpty computes the client IP instead.

7. Route filters: rewrite, rate limit, and cache

- 1 StripPrefix rewrites the path**
 application.yml:33
 StripPrefix=1 removes the first segment: /api/products -> /products.
Next: The internal target becomes http://localhost:8081/products.
- 2 RequestRateLimiter resolves a key**
 application.yml:34-39 -> RateLimitConfig.java:16-23
 For a valid JWT, the key is the Principal name. Redis applies 5 replenished tokens per second, burst capacity 10, and cost 1 per request.
Next: Allowed: continue. Empty bucket: 429.
- 3 Local response cache checks the request**
 application.yml:40
 For an eligible GET request, the local gateway cache looks for a saved response. Configuration is 30 seconds and 10 MB.
Next: Hit: return saved response. Miss: call product-service.
- 4 Gateway routes on a cache miss**
 SPRING: NettyRoutingFilter
 The gateway HTTP client sends GET http://localhost:8081/products.
Next: Product service DispatcherHandler resolves the controller.

Condition	Gateway result	ProductController.all() called?
Rate-limit token available, cache miss	Forward to product-service	Yes
Rate limit exceeded	429 Too Many Requests	No
Cache hit within gateway TTL	Return cached 200 response	No
Product service unavailable	Gateway routing failure, commonly 502/503	No successful call

Why repeated responses may have the same servedAt

On a gateway cache hit, ProductController does not create a new ProductResponse, so the previously cached servedAt value is returned.

8. Product service controller and response

1 **Product service receives GET /products**
ProductServiceApplication.java:9 -> SPRING DispatcherHandler
The separate product-service application is listening on port 8081. DispatcherHandler examines controller annotations.
Next: Find ProductController mapping.

2 **Class mapping matches**
ProductController.java:15-16
@RestController registers the class. @RequestMapping("/products") matches the rewritten path.
Next: Find a GET method with no extra path.

3 **all() is selected**
ProductController.java:24-26
@GetMapping plus the class mapping equals GET /products. Line 26 creates ProductResponse from PRODUCTS and Instant.now().
Next: Call cacheable(response).

4 **Static product list is used**
ProductController.java:18-22
The current demo has three in-memory products. There is no repository or database call.
Next: Build ResponseEntity.

5 **cacheable() returns HTTP response**
ProductController.java:38-41
Line 39 creates 200 OK. Line 40 adds Cache-Control public, max-age=300. Line 41 attaches the response body.
Next: Jackson serializes ProductResponse to JSON.

```
{
  "products": [
    {"id": 1, "name": "Mechanical Keyboard", "price": 89.99},
    {"id": 2, "name": "Wireless Mouse", "price": 39.99},
    {"id": 3, "name": "USB-C Dock", "price": 129.00}
  ],
  "servedAt": "2026-08-20T10:15:00Z"
}
```

Two cache settings

ProductController advertises public max-age=300 seconds in its response. The gateway route's LocalResponseCache is configured for 30 seconds and 10 MB.

9. Exact from-here-to-here trace

```

SecurityConfig.java:49-50
  -> registers JWT resource-server processing
SecurityConfig.java:51
  -> builds and stores SecurityWebFilterChain
REQUEST ARRIVES
  -> SPRING AuthenticationWebFilter.filter()
  -> SPRING Bearer token converter
  -> SPRING JwtReactiveAuthenticationManager.authenticate()
  -> decode() on object returned at SecurityConfig.java:82
  -> convert() on object returned at SecurityConfig.java:94
  -> SecurityConfig.java:45 authenticated() rule
  -> application.yml:31 product route predicate
  -> RequestLoggingFilter.java:26 filter()
  -> RequestLoggingFilter.java:37 chain.filter()
  -> application.yml:33 StripPrefix
  -> RateLimitConfig.java:16 resolver lambda
  -> RateLimitConfig.java:17 map(Principal::getName)
  -> application.yml:40 LocalResponseCache
  -> GET http://localhost:8081/products
  -> ProductController.java:16 class mapping
  -> ProductController.java:24-26 all()
  -> ProductController.java:38-41 cacheable()
  -> Gateway cache and RequestLoggingFilter.java:38-39
  -> Postman receives response

```

Useful debugger breakpoints

Breakpoint	What it proves
SecurityConfig.java:77	Factory runs at startup, not for every request.
RequestLoggingFilter.java:26	Authorized request reached Gateway filter processing.
RateLimitConfig.java:16	Resolver executes and can see the Principal.
ProductController.java:25	Gateway forwarded a cache-miss request to product-service.
ProductController.java:38	Controller is building the final 200 response.

Cache debugging tip

If ProductController.java:25 does not trigger on the second request but Postman still receives 200, the gateway likely returned a cached response. Wait more than 30 seconds or change the URL when testing the controller flow.

Final mental model

```

Decoder validates JWT
  -> Authorization permits authenticated user
  -> Route chooses product-service
  -> Filters log, limit, cache, and rewrite
  -> ProductController creates product JSON

```